

```

// DOT DROP バランス確認ツール
// 使い方: node balance.js
// index.html の /*ENGINE*/ ~ /*END*/ の物理とルールをそのまま読み込んで、
// 画面なしで大量に打ってバランスを測る。受け皿や玉数を変えたら必ず実行すること。

const fs = require('fs');
const html = fs.readFileSync(__dirname + '/index.html', 'utf8');
const engine = html.slice(html.indexOf('/*ENGINE*/'), html.indexOf('/*END*/'));
const num = (name, fallback) => {
  const m = html.match(new RegExp(name + '\\s*=\\s*(\\d+)'));
  return m ? +m[1] : fallback;
};
const START = num('START', 12), FEVER_AT = num('FEVER_AT', 120), FEVER_SHOTS = nu
const TRI_BONUS = 3;

const {E, LAYOUTS, CONV_LEN, STAGE_B, setLayout, pickGold, launch, launchVelocity
  new Function(engine + '\nreturn {E, LAYOUTS, CONV_LEN, STAGE_B, setLayout, pick

// 1回打って結果を返す。angleDeg: 0=右, 90=真上(下に引く), 180=左 / level: 1~5
function shoot(angleDeg, level) {
  let done = false;
  E.hooks.shotEnd = () => { done = true; };
  const a = angleDeg * Math.PI / 180, p = level / 5 * 130;
  launch(...launchVelocity(Math.cos(a) * p, Math.sin(a) * p));
  let t = 0, maxBalls = 1;
  while (!done && t < 40) { stepPhysics(1 / 360); t += 1 / 360; maxBalls = Math.m
  const tris = E.pegs.filter(q => q.kind === 'tri');
  const bonus = tris.length && tris.every(q => q.triHit) ? TRI_BONUS : 0;
  return {balls: E.shotPay + bonus, score: E.shotScore, maxBalls, stuck: !done};
}
const randomShot = () => [5 + Math.random() * 170, 1 + (Math.random() * 5 | 0)];

// ① ステージごとの「1回あたりの持ち玉の増減」(適当に打った場合)
console.log('\n① ステージ別: 1回あたりの持ち玉の増減 (コスト1を引いた値、適当打ち) ');
for (let st = 0; st < STAGE_B.length; st++) {
  E.stage = st; E.fever = false;
  const row = [];
  for (let k = 0; k < LAYOUTS; k++) {
    setLayout(k, false);
    let sum = 0; const N = 600;
    for (let i = 0; i < N; i++) { E.conveyor = Math.random() * CONV_LEN; pickGold
    row.push((sum / N - 1).toFixed(2));
  }
  console.log(` ステージ${st + 1}: 台ごと ${row.join(' ')} `);
}

```

```
}
```

```
// ② 一番得な打ち方を探す（うまい人が同じ打ち方を繰り返す想定）
```

```
console.log(`\n② ステージ1で一番得な打ち方（上位3つ）`);
```

```
E.stage = 0; E.fever = false;
```

```
const results = [];
```

```
for (let k = 0; k < LAYOUTS; k++) {
```

```
  setLayout(k, false);
```

```
  for (let lv = 1; lv <= 5; lv++) for (let a = 5; a <= 175; a += 10) {
```

```
    let sum = 0; const N = 80;
```

```
    for (let i = 0; i < N; i++) { E.conveyor = Math.random() * CONV_LEN; pickGold
```

```
    results.push({layout: k, angle: a, level: lv, net: sum / N - 1});
```

```
  }
```

```
}
```

```
results.sort((x, y) => y.net - x.net);
```

```
results.slice(0, 3).forEach(r => console.log(`  台${r.layout} 角度${r.angle}° パワ-
```

```
// ③ 1ゲームを通して何回打てるか
```

```
function game(strategy) {
```

```
  let money = START, gauge = 0, feverLeft = 0, shots = 0, board = 0, score = 0;
```

```
  E.stage = 0; E.fever = false; setLayout(0, false);
```

```
  E.hooks.hit = (p, b, n, f, kind, pts) => { if (!E.fever) gauge += pts; };
```

```
  while (money >= 1 && shots < 1000) {
```

```
    money--; shots++; E.conveyor = Math.random() * CONV_LEN;
```

```
    const r = strategy === 'random' ? shoot(...randomShot()) : shoot(strategy.ang
```

```
    money = Math.max(0, money + r.balls); score += r.score;
```

```
    if (E.fever) { if (--feverLeft <= 0) { E.fever = false; gauge = 0; } }
```

```
    else if (gauge >= FEVER_AT) { E.fever = true; feverLeft = FEVER_SHOTS; }
```

```
    if (++board >= PER_STAGE) { board = 0; E.stage++; setLayout((E.layout + 1) %
```

```
  }
```

```
  E.hooks.hit = () => {};
```

```
  return {shots, stage: E.stage + 1, score};
```

```
}
```

```
const median = arr => arr.slice().sort((a, b) => a - b)[arr.length >> 1];
```

```
console.log(`\n③ 1ゲームの長さ（持ち玉${START}スタート、各40ゲーム）`);
```

```
for (const [label, strat] of [['適当打ち', 'random'], ['一番得な打ち方', results[0]]])
```

```
  const g = Array.from({length: 40}, () => game(strat));
```

```
  console.log(`  ${label}: 打った回数 中央値${median(g.map(x => x.shots))}（最大${Mat
```

```
  ` / 到達ステージ 中央値${median(g.map(x => x.stage))} / スコア 中央値${median(g.ma
```

```
  }
```

```
console.log(`\n目安：1ゲーム25～35回。「一番得な打ち方」でも終わりが来ること。`);
```